ชั้น Symbolic ของ NeSy-Escape

คำอธิบายโค้ดทั้งหมด — แต่ละไฟล์ทำอะไร แต่ละส่วนใช้อย่างไร และดึงข้อมูลมาจากไหน

สาขา	draft1
ขอบเขต	app/symbolic/engine.py, app/schemas.py, data/rules.json, data/story.json, app/walkthrough.py และจุดที่ app/main.py เรียกใช้
จำนวนบรรทัด	engine.py 442 บรรทัด | schemas.py 102 | main.py 105
กฎในระบบ	15 ข้อ (R000–R014)
การทดสอบ	60 ข้อ (test_engine.py 50 ข้อ, test_neural.py 10 ข้อ)

เอกสารนี้อธิบายเฉพาะชั้น symbolic ส่วนชั้น neural (การเรียก Gemini) ถูกกล่าวถึงเพียงเท่าที่จำเป็นเพื่อให้เห็นว่าสองชั้นต่อกันตรงไหน

1. ภาพรวม แนวคิดของระบบสองชั้น

NeSy-Escape แบ่งการทำงานเป็นสองชั้นที่มีหน้าที่ต่างกันชัดเจน และห้ามก้าวก่ายกัน

ชั้น	หน้าที่	ใช้ LLM
Neural app/neural/	แปลข้อความภาษาไทยของผู้เล่นให้เป็นโครงสร้าง ParsedCommand ตอบคำถามว่า ผู้เล่นอยากทำอะไร กับสิ่งไหน	ใช้
Symbolic app/symbolic/	รับ ParsedCommand แล้วตัดสินว่าทำได้หรือไม่ ตามกฎใน rules.json ตอบคำถามว่า กติกาของเกมอนุญาตให้ทำหรือเปล่า	ไม่ใช้

หลักการสี่ข้อที่เขียนไว้บนหัวไฟล์ engine.py

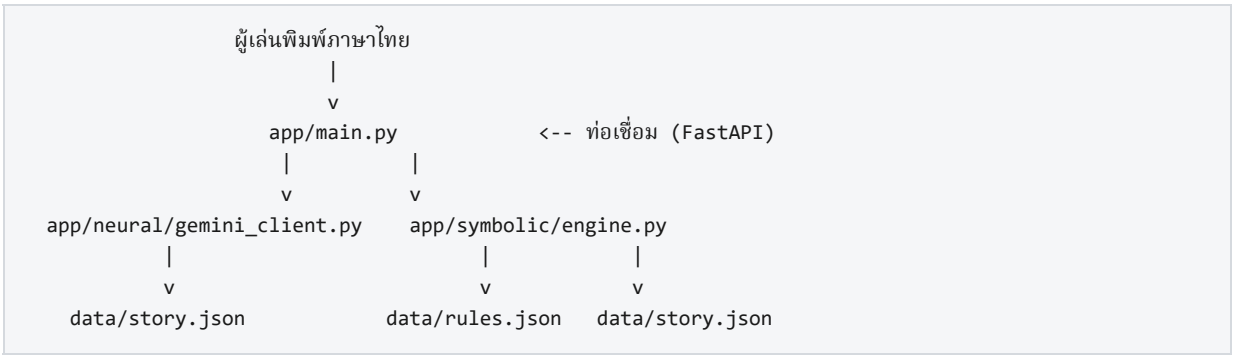
```
engine.py - Symbolic Rule Engine

หน้าที่: รับ ParsedCommand + GameState -> คืน Decision
หลักการที่ห้ามละเมิด:
  1) การตัดสินใจเกิดที่นี่ 100% ไม่มีการเรียก LLM ในไฟล์นี้
  2) reasoning_chain สร้างจากกฎที่ถูกตรวจจริงเท่านั้น (faithful by construction)
  3) กฎอยู่ใน rules.json ไม่ hard-code เงื่อนไขลงในโค้ด
  4) ไม่นำ confidence จาก LLM มาใช้ในการตัดสิน
```

ข้อ 2 สำคัญที่สุดสำหรับงานวิจัย คำว่า **faithful by construction** หมายความว่า คำอธิบายเหตุผลที่ระบบแสดงให้ผู้เล่นเห็น ไม่ได้ถูกเขียนขึ้นทีหลังเพื่อให้ดูสมเหตุสมผล แต่ทุกบรรทัดเกิดจากกฎที่โค้ดวิ่งผ่านจริง ถ้าอยากได้คำอธิบายเพิ่ม ต้องเพิ่มกฎ ไม่ใช่เพิ่มข้อความ

ข้อ 4 หมายความว่าต่อให้ LLM ตอบมาว่ามั่นใจ 0.99 หรือ 0.10 ชั้น symbolic ก็ตัดสินเหมือนเดิม
ค่านี้ถูกเก็บไว้เพื่อการวิเคราะห์เท่านั้น ไม่มีบรรทัดไหนใน engine.py อ่านมันเลย

2. แผนที่ไฟล์ ใครดึงอะไรมาจากไหน



ฝั่งซ้ายอ่าน story.json ไปใช้สร้าง prompt ส่วนฝั่งขวาอ่าน rules.json ซึ่งเก็บกฎ 15 ข้อ และ story.json ซึ่งเก็บโลกของเกม ทั้งสองฝั่งคุยกันด้วยโครงสร้างใน app/schemas.py เท่านั้น

ไฟล์	หน้าที่	ดึงอะไรมา
`app/schemas.py`	นิยามโครงสร้างข้อมูลกลาง 5 แบบ ที่สองชั้นใช้คุยกัน	pydantic
`app/symbolic/engine.py`	หัวใจของงาน ตัวตรวจกฎ 15 ตัว คลาส SymbolicEngine และฟังก์ชัน apply_changes	schemas.py rules.json story.json
`data/rules.json`	กฎทั้ง 15 ข้อ เก็บเป็นข้อมูล ไม่ใช่โค้ด	—
`data/story.json`	โลกของเกม ตู้รถ 6 ของ 21 คน 6 พร้อมคำบรรยายและเงื่อนไข	—
`app/walkthrough.py`	อ่านเส้นทางเฉลย 29 ขั้นจาก walkthrough.json ให้เทสใช้	walkthrough.json
`app/main.py`	รับคำสั่งจากเว็บ เรียก neural แล้วส่งต่อให้ symbolic	ทั้งสองชั้น

จุดที่ควรสังเกต engine.py **ไม่ import อะไรจาก app/neural เลย** จึงรันได้โดยไม่ต้องมี GEMINI_API_KEY และเทส 50 ข้อของมันรันได้แม้อินเทอร์เน็ตหลุด นี่คือการบังคับด้วยโครงสร้างว่าชั้นตัดสินไม่แตะ LLM จริง ไม่ใช่แค่ข้อตกลงกันปากเปล่า

3. app/schemas.py สัญญากลางระหว่างสองชั้น

ไฟล์นี้ไม่มีตรรกะใด ๆ มีแต่คำนิยามโครงสร้างข้อมูล (Pydantic) หน้าที่คือทำให้สองชั้นต่อกันผิดไม่ได้
ถ้าฝั่งหนึ่งส่งฟิลด์ผิดชื่อหรือผิดชนิด จะพังตั้งแต่ตอนตรวจ ไม่ใช่ไปพังกลางเกม

3.1 ActionType คำกริยาที่ระบบรู้จัก

```
class ActionType(str, Enum):
    EXAMINE     = "examine"      # ตรวจสอบวัตถุ/สถานที่/หลักฐาน
    TAKE        = "take"         # หยิบ/เก็บวัตถุเข้ากระเป๋า
    USE         = "use"          # ใช้ไอเทมกับเป้าหมาย
    UNLOCK      = "unlock"       # ปลดล็อกวัตถุ/สถานที่
    MOVE        = "move"         # เคลื่อนที่ไปยังตู้รถอื่น
    INTERROGATE = "interrogate"  # สอบปากคำผู้ต้องสงสัย
    ACCUSE      = "accuse"       # กล่าวหาฆาตกร (จบเกม)

    AMBIGUOUS   = "ambiguous"    # คำสั่งกำกวม ตีความได้หลายทาง
    UNKNOWN     = "unknown"      # นอกขอบเขต หรือตีความไม่ได้
```

สองค่าล่างสำคัญกว่าที่คิด มันคือช่องทางให้ชั้น neural บอกว่า ฉันไม่รู้ แทนที่จะเดาแล้วส่งคำสั่งมั่ว ๆ มาให้ชั้น symbolic ทำ

3.2 ParsedCommand ผลลัพธ์จากชั้น neural

ฟิลด์	ชนิด	ความหมาย
`action`	`ActionType`	กริยาที่สกัดได้
`target`	`str` หรือ `None`	id ภาษาอังกฤษของเป้าหมาย เช่น service_pantry
`instrument`	`str` หรือ `None`	id ของไอเทมที่ใช้ ใช้กับ action use เท่านั้น
`raw_text`	`str`	ข้อความไทยดิบที่ผู้เล่นพิมพ์ เก็บไว้ตรวจย้อนหลัง
`confidence`	`float 0` ถึง `1`	ความมั่นใจของ LLM เก็บเพื่อวิเคราะห์ ไม่ใช้ตัดสิน
`alternatives`	`list ของ str`	ตัวเลือก เมื่อ action เป็น ambiguous

มีเทสข้อหนึ่งคอยล็อกไว้ว่าโครงสร้างนี้ **ต้องไม่มีฟิลด์ชื่อ allowed, permitted, valid หรือ decision** เพราะถ้ามีเมื่อไหร่ แปลว่าชั้น neural เริ่มตัดสินแทน ซึ่งทำลายข้อสรุปของงานวิจัยทั้งหมด

3.3 GameState สถานะเกม ณ ขณะนั้น

```
class GameState(BaseModel):
    current_location: str            = "passenger_car"    # ตู้ที่ยืนอยู่
    inventory: list[str]             = []                 # ของที่ถือติดตัว
    discovered_evidence: list[str]   = []                 # หลักฐานที่พบแล้ว
    unlocked_objects: list[str]      = []                 # สิ่งที่เปิด/ตรวจแล้ว
    visited_locations: list[str]     = ["passenger_car"]  # ตู้ที่เคยไป
    interrogated_suspects: list[str] = []                 # คนที่สอบปากคำแล้ว
    elapsed_seconds: int             = 0                  # เวลาที่ใช้ไป
    turn_count: int                  = 0                  # จำนวนตา
```

เซิร์ฟเวอร์ **ไม่เก็บสถานะไว้เอง** ทุกครั้งที่ผู้เล่นส่งคำสั่ง ฝั่งเว็บจะแนบ GameState ทั้งก้อนมาด้วย
แล้วเซิร์ฟเวอร์ส่งก้อนใหม่กลับไป ข้อดีคือรันหลายเกมพร้อมกันได้โดยไม่ปนกัน

และการทดลองสามารถยัดสถานะที่ต้องการเข้าไปตรงกลางได้เลย ไม่ต้องเล่นไล่มาตั้งแต่ต้น

3.4 ReasoningStep และ Decision ผลลัพธ์จากชั้น symbolic

```
class ReasoningStep(BaseModel):
    rule_id: str        # รหัสกฎใน rules.json เช่น "R005_SAME_LOCATION"
    description: str    # คำอธิบายกฎภาษาไทย
    passed: bool        # ผ่านหรือไม่

class Decision(BaseModel):
    allowed: bool                          # อนุมัติหรือไม่
    message: str                           # ข้อความที่ผู้เล่นจะเห็น
    reasoning_chain: list[ReasoningStep]  # ลำดับกฎที่ถูกตรวจจริง
    violated_rule_id: str | None           # กฎที่ทำให้ถูกปฏิเสธ
    state_changes: dict                    # รายการเปลี่ยนแปลงสถานะ
```

reasoning_chain คือผลผลิตหลักของงานวิจัยนี้ มันไม่ใช่คำอธิบายที่ LLM แต่งขึ้น
แต่เป็นบันทึกว่าโค้ดวิ่งผ่านกฎข้อไหนบ้างและผลเป็นอย่างไร ตรวจสอบย้อนกลับไปที่ rules.json ได้ทุกบรรทัด

4. data/story.json โลกของเกม

ไฟล์นี้คือฉากทั้งหมด ถ้าอยากเปลี่ยนคดีใหม่ แก้ไฟล์นี้ไฟล์เดียวโดยไม่ต้องแตะโค้ด

คีย์บนสุด	ความหมาย
`title, briefing`	ชื่อคดีและบทเกริ่นที่ผู้เล่นเห็นตอนเปิดเกม
`time_limit_seconds`	กรอบเวลาทั้งคดี ปัจจุบัน 3600 วินาที เท่ากับ 60 นาที
`seconds_per_turn`	หนึ่งตากินเวลาเท่าไร ปัจจุบัน 60 วินาที ทำให้เล่นได้ 60 ตา
`locations`	ตู้รถ 6 ตู้
`objects`	สิ่งของและหลักฐาน 21 ชิ้น
`suspects`	ผู้เกี่ยวข้อง 6 คน
`culprit, framed, coerced`	เฉลยของฉาก ใช้เขียนเทส ไม่ได้ส่งให้ผู้เล่น

4.1 ฟิลด์ใน locations

ฟิลด์	มีกี่รายการ	ใครอ่าน	ผลที่เกิด
`name_th`	6/6	หลายจุด	ชื่อไทยที่ผู้เล่นเห็น
`description`	6/6	look_around	คำบรรยายตู้
`locked`	6/6	R004	ตู้นี้ล็อกอยู่หรือไม่
`requires`	2/6	R004	ต้องมีของชิ้นไหนในกระเป๋าจึงจะเข้าได้

4.2 ฟิลด์ใน objects

ฟิลด์	มีกี่รายการ	ใครอ่าน	ผลที่เกิด
`name_th, description`	21/21	_success_message	ข้อความที่คืนเมื่อตรวจสำเร็จ
`type`	21/21	_state_changes	ถ้าเป็น evidence การตรวจจะบันทึกเข้า discovered_evidence
`takeable`	21/21	R006	หยิบติดตัวได้หรือไม่
`location`	20/21	R005	อยู่ตู้ไหน ชิ้นที่ไม่มีคือ crew_key เพราะได้จากการคุยกับคน
`hidden_in`	2/21	R007	ซ่อนอยู่ในอะไร ต้องตรวจสิ่งนั้นก่อน
`requires_evidence`	8/21	R008	ต้องพบหลักฐานชิ้นไหนก่อนจึงจะเห็นชิ้นนี้
`requires_suspect`	2/21	R009	ต้องสอบปากคำใครก่อน
`supports`	17/21	R013	หลักฐานชิ้นนี้ค้ำเสาไหนของใคร
`exculpates`	1/21	R012	หลักฐานชิ้นนี้พิสูจน์ว่าใครถูกบังคับ

5. data/rules.json กฎทั้ง 15 ข้อ

กฎถูกเก็บเป็น **ข้อมูล ไม่ใช่โค้ด** เพิ่มหรือแก้กฎจึงทำได้โดยไม่ต้องแก้ engine.py ตราบใดที่ยังใช้ตัวตรวจเดิม แต่ละข้อมี 6 ฟิลด์

```
{
  "id": "R005_SAME_LOCATION",
  "description": "ต้องอยู่ในตู้รถเดียวกับสิ่งหรือคนที่ต้องการเข้าถึง",
  "applies_to": ["examine", "take", "use", "interrogate"],
  "check": "in_same_location",
  "params": {},
  "message_fail": "สิ่งหรือคนที่ระบุไม่ได้อยู่ในตู้รถนี้  ต้องเดินไปหาก่อน"
}
```

ฟิลด์	ใช้ทำอะไร
`id`	รหัสกฎ โผล่ใน reasoning_chain และ violated_rule_id ให้ตรวจย้อนได้
`description`	คำอธิบายภาษาไทย เอาไปแสดงในสายเหตุผลตรง ๆ
`applies_to`	กฎนี้ใช้กับ action ไหนบ้าง ใส่ดอกจันแปลว่าทุก action
`check`	ชื่อฟังก์ชันตรวจใน engine.py ผูกกันผ่านตาราง CHECKERS
`params`	ค่าที่ส่งให้ฟังก์ชันตรวจ ใช้ปรับพฤติกรรมโดยไม่แก้โค้ด
`message_fail`	ข้อความที่ผู้เล่นเห็นเมื่อกฎนี้ไม่ผ่าน

5.1 กฎทั้งหมด เรียงตามลำดับที่ถูกตรวจจริง

ลำดับ	รหัส	ใช้กับ action	ตรวจอะไร
1	`R000_KNOWN_ACTION`	*	คำสั่งต้องตีความได้และอยู่ในขอบเขตของสถานการณ์
2	`R014_TIME_LIMIT`	*	การกระทำต้องอยู่ภายในกรอบเวลาของการสืบสวน
3	`R001_TARGET_EXISTS`	`examine, take, use, unlock`	เป้าหมายของคำสั่งต้องมีอยู่จริงบนขบวนรถ
4	`R002_LOCATION_EXISTS`	`move`	จุดหมายของการเคลื่อนที่ต้องเป็นตู้รถที่มีอยู่จริง
5	`R003_SUSPECT_EXISTS`	`interrogate, accuse`	ผู้ถูกสอบปากคำหรือถูกกล่าวหาต้องเป็นบุคคลบนขบวนรถ
6	`R004_LOCATION_ACCESS`	`move`	ตู้รถที่ล็อกอยู่ต้องมีกุญแจที่ถูกต้องจึงจะเข้าได้
7	`R005_SAME_LOCATION`	`examine, take, use, interrogate`	ต้องอยู่ในตู้รถเดียวกับสิ่งหรือคนที่ต้องการเข้าถึง
8	`R006_TAKEABLE`	`take`	เก็บได้เฉพาะสิ่งที่เคลื่อนย้ายได้
9	`R007_CONTAINER_OPENED`	`examine, take, use`	สิ่งที่ซ่อนอยู่ต้องเปิดที่ซ่อนก่อนจึงจะเข้าถึงได้

10	R008_EVIDENCE_PREREQ	examine, take	หลักฐานบางชิ้นจะเข้าถึงได้ก็ต่อเมื่อพบหลักฐานที่นำไปสู่มันก่อน
11	R009_TESTIMONY_PREREQ	examine, take	หลักฐานที่ได้จากคำให้การ ต้องสอบปากคำบุคคลนั้นก่อน
12	R010_INTERROGATE_PREREQ	interrogate	การสอบปากคำต้องมีหลักฐานเพียงพอที่จะตั้งคำถามได้
13	R011_INSTRUMENT_OWNED	use	สิ่งที่นำมาใช้ต้องอยู่ในครอบครองก่อน
14	R012_NO_ACCUSE_COERCED	accuse	ห้ามกล่าวหาบุคคลที่มีหลักฐานยืนยันว่าถูกบังคับ
15	R013_ACCUSE_FOUR_PILLARS	accuse	การกล่าวหาต้องมีหลักฐานครบทั้งสี่ด้าน คือ แรงจูงใจ วิธีการ โอกาส และหลักฐานสนับสนุน

ลำดับในไฟล์คือลำดับการตรวจ และมีผลจริง R014 เรื่องหมดเวลาถูกย้ายมาไว้อันดับสอง เพราะตอนแรกวางไว้ท้ายสุด ทำให้ผู้เล่นที่หมดเวลาแล้วกดกล่าวหา จะได้ข้อความว่า หลักฐานไม่ครบสี่ด้าน แทนที่จะเป็น หมดเวลาแล้ว ซึ่งเป็นคำอธิบายที่ผิดความจริง ตอนนี้มีเทสล็อกลำดับนี้ไว้ไม่ให้ใครสลับกลับ

6. engine.py ตอนที่ 1 ตัวตรวจกฎ 15 ตัว

โค้ดครึ่งบนของ engine.py คือฟังก์ชันตรวจกฎ ทุกตัวมีหน้าตาเหมือนกันหมด

```
def _ชื่อ(cmd, state, world, p):
    ...
    return ผ่านหรือไม่, "รายละเอียดที่จะแสดงในสายเหตุผล"
```

พารามิเตอร์	คืออะไร
`cmd`	ParsedCommand ที่ชั้น neural ส่งมา
`state`	GameState ปัจจุบัน
`world`	ข้อมูลทั้งก้อนจาก story.json
`p`	ค่า params ของกฎข้อนั้นจาก rules.json

ทุกตัวเป็นฟังก์ชันบริสุทธิ์ คือไม่แก้ไขอะไรเลย แค่ตอบว่าผ่านหรือไม่ผ่าน การเปลี่ยนสถานะเป็นหน้าที่ของฟังก์ชันอื่นทั้งหมด

6.1 ฟังก์ชันตรวจทั้ง 15 ตัว

ฟังก์ชัน	กฎ	ตรรกะโดยย่อ	อ่านจาก
`_action_is_known`	R000	ถ้า action เป็น unknown หรือ ambiguous ให้ตก	cmd
`_target_exists`	R001	target ต้องอยู่ใน objects หรือ locations รับชื่อตู้ด้วย เพราะ ดูรอบ ๆ คือ examine ตู้	world
`_location_exists`	R002	target ต้องอยู่ใน locations	world
`_suspect_exists`	R003	target ต้องอยู่ใน suspects	world
`_location_unlocked`	R004	ถ้าตู้ locked ต้องมี requires อยู่ใน inventory	world, state
`_in_same_location`	R005	เทียบ location ของเป้าหมายกับ current_location ค้นทั้ง objects และ suspects	world, state
`_target_is_takeable`	R006	ของต้องมี takeable เป็น true	world
`_container_unlocked`	R007	ถ้ามี hidden_in ตัวที่ซ่อนต้องอยู่ใน unlocked_objects	world, state
`_evidence_prerequisites_met`	R008	ทุก id ใน requires_evidence ต้องอยู่ใน discovered_evidence	world, state
`_testimony_prerequisite_met`	R009	requires_suspect ต้องอยู่ใน interrogated_suspects	world, state
`_suspect_available`	R010	คนนี้ต้องมีหลักฐานตั้งประเด็นครบ และสอบคนก่อนหน้าแล้ว	world, state
`_instrument_in_inventory`	R011	instrument ต้องอยู่ใน inventory	state
`_not_exculpated`	R012	ห้ามกล่าวหาคนที่มีหลักฐาน exculpates ชี้ว่าถูกบังคับ	world, state
`_four_pillars`	R013	เสาทั้งสี่ต้องมีหลักฐานอย่างน้อยด้านละหนึ่งชิ้น	world, state

`_within_time_limit`	R014	elapsed_seconds ต้องไม่เกิน time_limit_seconds	world, state

6.2 ตัวอย่างโค้ดจริง _in_same_location

ตัวที่ซับซ้อนที่สุด เพราะรับได้ทั้งตู้ ของ และคน

```
def _in_same_location(cmd, state, world, p):
    if cmd.target in world["locations"]:
        # มองรอบๆ ได้เฉพาะตู้ที่ยืนอยู่ ตู้อื่นต้องเดินไปก่อน
        if cmd.target != state.current_location:
            return False, f"ต้องเดินไปที่ '{cmd.target}' ก่อนจึงจะมองเห็นได้"
        return True, ""
    # ค้นทั้งของและคน คนก็ต้องอยู่ตู้เดียวกันถึงจะสอบปากคำได้ ตะโกนข้ามขบวนไม่ได้
    entity = world["objects"].get(cmd.target) or world["suspects"].get(cmd.target, {})
    where = entity.get("location")
    if where is None:
        return True, ""
    if cmd.target in state.inventory:
        return True, "อยู่ในครอบครองแล้ว"
    if where != state.current_location:
        return False, f"อยู่ที่ '{where}' แต่ผู้สืบสวนอยู่ที่ '{state.current_location}'"
    return True, ""
```

บรรทัด `where is None` คือทางผ่านของ crew_key ซึ่งเป็นของชิ้นเดียวที่ไม่มีที่อยู่ เพราะได้จากการคุยกับคน ไม่ได้วางอยู่ที่ไหน

6.3 ตัวอย่างโค้ดจริง _four_pillars กฎที่ทำให้เกมจบได้

```
def _collect_pillars(target, state, world):
    found = {k: [] for k in PILLARS}
    for ev_id in state.discovered_evidence:
        ev = world["objects"].get(ev_id, {})
        pillar = ev.get("supports", {}).get(target)
        if pillar in found:
            found[pillar].append(ev_id)
    return found

def _four_pillars(cmd, state, world, p):
    required = p.get("required", list(PILLARS))
    found = _collect_pillars(cmd.target, state, world)
    missing = [k for k in required if not found[k]]
    if missing:
        return False, f"ขาดหลักฐานด้าน: {', '.join(missing)}"
    summary = "; ".join(f"{k}={len(found[k])}" for k in required)
    return True, summary
```

สังเกตว่ามันไล่จาก `state.discovered_evidence` คือของที่ **ผู้เล่นหาเจอจริง** ไม่ใช่ของที่มีอยู่ในฉาก ผู้เล่นจึงต้องออกไปหาเอง และคำว่าครบสี่ด้านหมายถึงมีอย่างน้อยด้านละหนึ่งชิ้น ไม่ได้ต้องครบทุกชิ้น ทำให้มีหลายเส้นทางที่ชนะได้

รายชื่อเสาถูกส่งมาจาก params ใน rules.json ไม่ได้เขียนตายไว้ในโค้ด ถ้าอยากทดลองว่าใช้แค่สามเสาจะต่างกันไหม แก้ไฟล์ JSON ได้เลย

7. engine.py ตอนที่ 2 หัวใจของการตัดสิน

ตาราง CHECKERS คือจุดที่ผูกกฎในไฟล์ JSON เข้ากับฟังก์ชันในโค้ด ฟิลด์ check ของกฎต้องตรงกับคีย์ในตารางนี้ ถ้าสะกดผิดระบบจะโยน error ทันทีที่เจอกฎนั้น ไม่ใช่เงียบ ๆ ข้ามไป

```
CHECKERS: Dict[str, Callable] = {
    "action_is_known": _action_is_known,
    "target_exists": _target_exists,
    ...
    "within_time_limit": _within_time_limit,
}
```

7.1 __init__ และ _applicable

```
class SymbolicEngine:
    def __init__(self, rules_path=None, story_path=None):
        self.rules = json.loads(rules_path.read_text(...))["rules"]
        self.world = json.loads(story_path.read_text(...))

    def _applicable(self, action):
        return [r for r in self.rules
                if "*" in r.get("applies_to", []) or action.value in r.get("applies_to", [])]
```

โหลดไฟล์ทั้งสองครั้งเดียวตอนสร้างวัตถุ แล้วเก็บไว้ในหน่วยความจำ การรับ path เข้ามาได้ทำให้เทสสร้างโลกจำลองของตัวเองได้โดยไม่แตะไฟล์จริง ส่วน _applicable คือตัวกรองว่า action นี้ต้องผ่านกฎข้อไหนบ้าง เช่น move ผ่าน 4 ข้อ ส่วน accuse ผ่าน 5 ข้อ

7.2 decide() วงจรหลัก

```
def decide(self, cmd, state) -> Decision:
    chain = []
    for rule in self._applicable(cmd.action):
        checker = CHECKERS.get(rule["check"])
        if checker is None:
            raise ValueError(f"ไม่พบ checker '{rule['check']}' ของกฎ {rule['id']}")

        passed, detail = checker(cmd, state, self.world, rule.get("params", {}))
        desc = f"{rule['description']} ({detail})" if detail else rule["description"]
        chain.append(ReasoningStep(rule_id=rule["id"], description=desc, passed=passed))

        # หยุดที่กฎแรกที่ไม่ผ่าน  เพราะกฎถัดไปอาจสมมติว่ากฎก่อนหน้าผ่านแล้ว
        if not passed:
            ...
            return Decision(allowed=False, message=message, reasoning_chain=chain,
                            violated_rule_id=rule["id"], state_changes={})

    return Decision(allowed=True, message=self._success_message(cmd, state),
                    reasoning_chain=chain, violated_rule_id=None,
                    state_changes=self._state_changes(cmd))
```

ทั้งฟังก์ชันมีแค่ 20 บรรทัด และนี่คือจุดที่ **reasoning_chain ถูกสร้างขึ้นระหว่างทาง** ไม่ได้เขียนทีหลัง ทุก ReasoningStep ถูกเพิ่มเข้าลิสต์ทันทีหลังเรียกตัวตรวจเสร็จ จึงเป็นไปไม่ได้เลยที่สายเหตุผลจะไม่ตรงกับสิ่งที่ระบบทำจริง

การหยุดที่กฎแรกที่ไม่ผ่านเป็นเรื่องจำเป็น ไม่ใช่แค่เรื่องความเร็ว เพราะกฎข้อหลังสมมติว่าข้อหน้าผ่านแล้ว เช่น R005 จะไปอ่าน location ของเป้าหมาย ซึ่งจะพังถ้า R001 ยังไม่ได้ยืนยันว่าเป้าหมายมีอยู่จริง

7.3 การปรับข้อความให้ผู้เล่นไม่ตัน

ในกิ่งที่กฎไม่ผ่าน มีการปรับข้อความสามกรณี ทั้งหมดมาจากการให้คนอื่นลองเล่นจริง แล้วพบว่าอ่านแล้วไม่รู้ว่าต้องทำอะไรต่อ

กรณี	ปัญหาเดิม	แก้เป็น
`action = ambiguous`	ได้ข้อความว่า ระบบไม่เข้าใจคำสั่งนี้ เหมือนกรณี unknown	บอกว่าคำสั่งกว้างไป ให้ระบุให้เจาะจง หรือพิมพ์ ดูรอบๆ
`action = unknown`	บอกแค่ว่าไม่เข้าใจ ไม่บอกว่าทำอะไรได้บ้าง	ต่อท้ายด้วยรายการสิ่งที่ทำได้ทั้งหมด
`R005` ไม่ผ่าน	บอกว่าต้องเดินไปหาก่อน แต่ไม่บอกว่าตู้ไหน ทั้งที่เอนจินรู้อยู่แล้ว	ต่อท้ายชื่อไทยของตู้ปลายทาง เช่น มารา วอสส์ อยู่ที่ตู้โดยสาร

จุดสำคัญคือการแก้เหล่านี้แตะแค่ฟิลด์ message ซึ่งเป็นข้อความสำหรับมนุษย์ **ไม่แตะ reasoning_chain** สายเหตุผลจึงยังเป็นบันทึกทางเทคนิคที่ซื่อตรงเหมือนเดิม

8. engine.py ตอนที่ 3 การเปลี่ยนสถานะ

การตัดสินกับการเปลี่ยนสถานะถูกแยกออกจากกันโดยตั้งใจ decide() แค่บอกว่า **ควรเปลี่ยนอะไร** ส่วนการเปลี่ยนจริงเกิดที่ apply_changes() ซึ่งอยู่นอกคลาส

8.1 _state_changes() แปลง action เป็นใบสั่งเปลี่ยนสถานะ

action	state_changes ที่คืน	หมายเหตุ
move	{"move_to": target}	—
take	{"inventory_add": target} บวก discovered_evidence_add ถ้า type เป็น evidence	—
examine	{} ถ้าเป้าหมายเป็นตู้ เพราะดูรอบ ๆ ไม่เปลี่ยนอะไร {"unlocked_add": target} ถ้าเป็นของ บวก discovered_evidence_add ถ้า type เป็น evidence	ตรวจของเท่ากับเปิดของนั้น ทำให้ของที่ hidden_in มันโผล่ตามมา
unlock	{"unlocked_add": target}	—
interrogate	{"interrogated_add": target} บวก inventory_add ถ้าคนนั้นมี grants_object	ทางเดียวที่จะได้ crew_key
accuse, use	{}	จบเกม หรือยังไม่มีผลต่อสถานะ

8.2 apply_changes() ผู้เปลี่ยนสถานะตัวจริง

```
def apply_changes(state: GameState, changes: dict) -> GameState:
    s = state.model_copy(deep=True)
    mapping = {
        "inventory_add": s.inventory,
        "discovered_evidence_add": s.discovered_evidence,
        "unlocked_add": s.unlocked_objects,
        "interrogated_add": s.interrogated_suspects,
    }
    for key, lst in mapping.items():
        v = changes.get(key)
        if v and v not in lst:
            lst.append(v)
    dest = changes.get("move_to")
    if dest:
        s.current_location = dest
        if dest not in s.visited_locations:
            s.visited_locations.append(dest)
    s.turn_count += 1
    s.elapsed_seconds += SECONDS_PER_TURN
    return s
```

คืน state ก้อนใหม่เสมอ ไม่แก้ของเดิม ทำให้ยิงคำสั่งชุดเดิมซ้ำกี่รอบก็ได้ผลเหมือนกัน
ซึ่งจำเป็นมากสำหรับการทดลองที่ต้องรันซ้ำได้

สองบรรทัดสุดท้ายคือจุดที่ต้องอธิบายเป็นพิเศษ เวลาเดินที่นี่ **ไม่ได้เดินผ่าน state_changes**
ทั้งที่ดูเหมือนควรจะเป็นแบบนั้น เหตุผลอยู่ในคอมเมนต์ของโค้ดเอง

```
เวลาเดินทุกตาโดยไม่ขึ้นกับว่าใครเป็นผู้ตัดสิน จงใจให้อยู่ตรงนี้ไม่ใช่ใน state_changes
เพราะถ้าปล่อยให้คำตัดสินเป็นตัวกำหนด โหมด A (LLM ตัดสินเอง) จะคุมเวลาของตัวเองได้
แล้วสองโหมดจะเดินด้วยนาฬิกาคนละเรือน เทียบกันไม่ได้
```

นี่เป็นการตัดสินใจเชิงการทดลอง ไม่ใช่เชิงเกม การเปรียบเทียบสองโหมดจำเป็นต้องให้
ทุกอย่างเหมือนกันหมดยกเว้นตัวแปรเดียวคือใครเป็นคนตัดสิน ถ้าเวลาเดินไม่เท่ากัน ผลที่ได้ก็เทียบกันไม่ได้

ค่า SECONDS_PER_TURN อ่านมาจาก story.json ตอนโหลดโมดูล ไม่ได้เขียนตัวเลขไว้ในโค้ด

9. engine.py ตอนที่ 4 ข้อความที่ผู้เล่นได้เห็น

เกมนี้เล่นด้วยการพิมพ์ล้วน ไม่มีปุ่มให้กด ผู้เล่นจึงรู้จักโลกได้จากข้อความที่ระบบตอบกลับเท่านั้น ส่วนนี้จึงสำคัญพอ ๆ กับตัวกฎ

9.1 _success_message() เลือกข้อความตอบกลับ

action	คืนอะไร
`examine ตู้`	เรียก look_around() ของตู้นั้น
`examine ของ`	ฟิลด์ description ของ object นั้นจาก story.json
`interrogate`	ฟิลด์ testimony ของ suspect นั้น
`move`	look_around() ของตู้ปลายทาง โดยใช้สำเนา state ที่ย้ายแล้ว
`accuse`	เรียก closing_argument()
`take, unlock, use`	ข้อความสำเร็จรูปสั้น ๆ เช่น เก็บ...ไว้เป็นหลักฐาน

ข้อสังเกต ข้อความส่วนใหญ่คืน **เนื้อความจริงจาก story.json** ไม่ใช่ประโยคยืนยันลอย ๆ
เพราะถ้าตอบแค่ว่าตรวจสอบเรียบร้อย ผู้เล่นจะไม่ได้ข้อมูลอะไรเลยและเล่นต่อไม่ได้

9.2 look_around() รายการสิ่งที่มองเห็นได้ตอนนี้

```
here = [oid for oid, o in self.world["objects"].items()
        if o.get("location") == loc_id and oid not in state.inventory]
visible = [oid for oid in here if self.decide(
    ParsedCommand(action=ActionType.EXAMINE, target=oid, raw_text="", confidence=0.0),
    state,
).allowed]
```

นี่คือเทคนิคสำคัญของไฟล์นี้ **look_around ไม่ได้มีรายการที่เขียนมือไว้ต่างหาก แต่ถามจากกฎชุดเดียวกับที่ใช้ตัดสินจริง** โดยลองเรียก decide() ด้วยคำสั่ง examine สมมติ
ของที่ยังซ่อนอยู่หรือยังไม่มีหลักฐานนำทาง จะตกกฎ R007 หรือ R008 แล้วไม่ถูกแสดง

ผลคือ **สิ่งที่ผู้เล่นเห็น กับสิ่งที่ผู้เล่นทำได้ ตรงกันเสมอโดยอัตโนมัติ** ถ้าเขียนรายการแยกไว้
วันหนึ่งกฎเปลี่ยนแล้วลืมแก้รายการ ผู้เล่นจะเห็นของที่ตรวจไม่ได้ หรือตรวจได้แต่มองไม่เห็น

ส่วนรายชื่อคนที่อยู่ในตู้นี้จะแสดง **ทุกคน** ที่อยู่ ไม่ว่าจะสอบปากคำได้แล้วหรือยัง เพราะถ้าไม่บอกว่ามีใครอยู่
ผู้เล่นจะไม่มีทางรู้ว่าสอบใครได้ ส่วนคนที่ยังคุยไม่ได้ กฎ R010 จะเป็นคนบอกเองว่าต้องหาหลักฐานอะไรก่อน

9.3 offerable() กันไม่ให้เกมเฉลยตัวเอง

ฟังก์ชันนี้ตอบคำถามว่า เอ่ยชื่อสิ่งนี้ให้ผู้เล่นเห็นได้ไหม โดยไม่เฉลยของที่เขายังหาไม่เจอ app/main.py เรียกมันมากรอง alternatives ก่อนส่งกลับ

```
def offerable(self, target, state) -> bool:
    obj = self.world["objects"].get(target)
    if obj is None:
        return True                      # ตู้รถหรือคน เอ่ยได้เสมอ
    if target in state.inventory or target in state.discovered_evidence:
        return True                      # ถือหรือเจอแล้ว
    return obj.get("location") == state.current_location and self.decide(
```

```
        ParsedCommand(action=ActionType.EXAMINE, target=target, ...), state,
    ).allowed                              # อยู่ตรงหน้าและมองเห็นได้
```

ที่มาของฟังก์ชันนี้ ผู้เล่นพิมพ์ว่า ปลดทุกอย่าง ตั้งแต่ตาแรก แล้วชั้น neural ซึ่งเห็นตารางทั้งโลกใน prompt แต่ไม่รู้ว่าผู้เล่นเดินไปถึงไหน ตอบกลับมาเป็นรายชื่อของ 4 ชิ้นในตู้ที่ยังไม่เคยไป รวมถึง crew_key ที่ต้องคุยกับคนถึงจะได้ เท่ากับเกมเฉลยตัวเอง

แก้ที่ backend ไม่ใช่ที่ prompt เพราะกันได้แน่นอนกว่า ทดสอบได้โดยไม่ต้องเรียก LLM และคุมทุกหน้าเว็บที่จะมาต่อทีหลัง สังเกตว่ามันใช้ decide() ถามกฎเหมือนที่ look_around ทำ ของที่ซ่อนอยู่จึงไม่หลุด

9.4 closing_argument() ฉากจบที่ประกอบจากหลักฐานจริง

```
sus = self.world["suspects"].get(suspect_id, {})
found = _collect_pillars(suspect_id, state, self.world)

lines = [f"คุณเรียกทุกคนมารวมกันที่ตู้เสบียง แล้วชี้ไปที่{sus.get('name_th', suspect_id)}", ""]
for pillar in PILLARS:
    names = [self.world["objects"][e].get("name_th", e) for e in found[pillar]]
    lines.append(f"  {label[pillar]}")
    for n in names:
        lines.append(f"    - {n}")
```

คำแถลงปิดคดี **ไม่ใช่ข้อความสำเร็จรูป** ทุกบรรทัดอ้างชิ้นหลักฐานที่อยู่ใน discovered_evidence จริง ๆ ถ้าผู้เล่นสองคนเก็บหลักฐานคนละชุด ฉากจบจะออกมาคนละแบบ แม้จะกล่าวหาคนเดียวกันก็ตาม

นี่คือหลักการเดียวกับ reasoning_chain คือ **คำอธิบายต้องมาจากสิ่งที่เกิดขึ้นจริง ไม่ใช่ข้อความที่เขียนรอไว้** ต่างกันแค่ reasoning_chain อธิบายว่าทำไมระบบตัดสินแบบนั้น ส่วน closing_argument อธิบายว่าทำไมผู้ต้องสงสัยคนนี้ผิด

9.5 pillar_status() สำหรับ UI และการวิเคราะห์

คืนความคืบหน้าสี่เสาของผู้ต้องสงสัยที่ระบุ มีคอมเมนต์กำกับไว้ว่าใช้สำหรับ UI และการวิเคราะห์ผล ไม่ใช่ส่วนของการตัดสิน เพื่อกันไม่ให้ใครเผลอเอาไปใช้ในเส้นทางตัดสินภายหลัง ในสาขา draft1 ตัว HUD ที่เคยแสดงข้อมูลนี้ถูกถอดออกแล้ว เพราะมันแอบบอกรายชื่อผู้ต้องสงสัยที่ผู้เล่นยังไม่ควรรู้

10. app/main.py จุดที่สองชั้นมาต่อกัน

```
@app.post("/command", response_model=CommandResponse)
def handle_command(req: CommandRequest):
    # ขั้นที่ 1: NEURAL LAYER
    try:
        parsed = interpret(req.text, req.state)
    except Exception as e:
        raise HTTPException(status_code=502, detail=f"Neural Layer ตีความล้มเหลว: {e}")

    # ขั้นที่ 2: SYMBOLIC LAYER
    decision = engine.decide(parsed, req.state)

    # ขั้นที่ 3: STATE UPDATE
    new_state = apply_changes(req.state, decision.state_changes) if decision.allowed else req.state

    # กรองตัวเลือกไม่ให้เฉลยของที่ผู้เล่นยังหาไม่เจอ
    parsed.alternatives = [a for a in parsed.alternatives
                           if engine.offerable((a.split() or [""])[-1], req.state)]

    return CommandResponse(parsed=parsed, decision=decision, new_state=new_state)
```

ทั้งเกมมีทางเข้าทางเดียวคือฟังก์ชันนี้ และมันสั้นแค่นี้ ลำดับสามขั้นบังคับว่า neural ต้องทำงานเสร็จก่อนเสมอ และ symbolic ไม่เคยเห็นข้อความดิบของผู้เล่นเลย เห็นแค่ ParsedCommand ที่ถูกแปลงแล้ว

บรรทัด `if decision.allowed` คือจุดบังคับว่า **คำสั่งที่ถูกปฏิเสธจะไม่เปลี่ยนสถานะใด ๆ เลย** รวมถึงไม่กินเวลาด้วย ผู้เล่นพิมพ์ผิดจึงไม่เสียตาฟรี

อีกสองปลายทางที่ใช้ engine โดยตรง

ปลายทาง	ใช้อะไรจาก engine
`GET /health`	len(engine.rules) และจำนวน objects เพื่อยืนยันว่าโหลดไฟล์ข้อมูลสำเร็จ
`GET /scene`	engine.world ทั้งก้อน แต่คืนเฉพาะ id กับชื่อไทย ไม่ส่ง description หรือเงื่อนไขออกไป

11. ตัวอย่างการทำงานจริง

ทั้งสองตัวอย่างนี้เป็นผลลัพธ์ที่รันออกมาจริงจากโค้ดชุดปัจจุบัน ไม่ได้เขียนขึ้นเอง

11.1 คำสั่งที่ถูกปฏิเสธ

```
cmd   = ParsedCommand(action='move', target='electrical_compartment')
state = current_location='passenger_car', inventory=[]

reasoning_chain ที่ได้ 4 ขั้น
  [ผ่าน]     R000_KNOWN_ACTION       คำสั่งต้องตีความได้และอยู่ในขอบเขตของสถานการณ์
  [ผ่าน]     R014_TIME_LIMIT         การกระทำต้องอยู่ภายในกรอบเวลาของการสืบสวน
  [ผ่าน]     R002_LOCATION_EXISTS    จุดหมายของการเคลื่อนที่ต้องเป็นตู้รถที่มีอยู่จริง
  [ไม่ผ่าน]  R004_LOCATION_ACCESS    ตู้รถที่ล็อกอยู่ต้องมีกุญแจที่ถูกต้องจึงจะเข้าได้
                                       (ต้องมี 'crew_key' จึงจะเข้าได้)

allowed       = False
violated      = R004_LOCATION_ACCESS
state_changes = {}
```

สังเกตว่าสายเหตุผลหยุดที่ R004 ทันที กฎที่เหลือไม่ถูกตรวจเลย และข้อความในวงเล็บ (ต้องมี 'crew_key' จึงจะเข้าได้) คือค่า detail ที่ตัวตรวจคืนกลับมา ไม่ใช่ข้อความที่เขียนไว้ล่วงหน้า

11.2 คำสั่งที่ผ่าน และผลต่อสถานะ

```
cmd   = ParsedCommand(action='interrogate', target='mara_voss')
state = current_location='passenger_car', inventory=[]

reasoning_chain ที่ได้ 5 ขั้น
  [ผ่าน] R000_KNOWN_ACTION         คำสั่งต้องตีความได้และอยู่ในขอบเขตของสถานการณ์
  [ผ่าน] R014_TIME_LIMIT           การกระทำต้องอยู่ภายในกรอบเวลาของการสืบสวน
  [ผ่าน] R003_SUSPECT_EXISTS       ผู้ถูกสอบปากคำหรือถูกกล่าวหาต้องเป็นบุคคลบนขบวนรถ
  [ผ่าน] R005_SAME_LOCATION        ต้องอยู่ในตู้รถเดียวกับสิ่งหรือคนที่ต้องการเข้าถึง
  [ผ่าน] R010_INTERROGATE_PREREQ   การสอบปากคำต้องมีหลักฐานเพียงพอที่จะตั้งคำถามได้

allowed       = True
state_changes = {'interrogated_add': 'mara_voss', 'inventory_add': 'crew_key'}

หลังส่งเข้า apply_changes()
  inventory     []  ->  ['crew_key']
  interrogated  []  ->  ['mara_voss']
  turn_count    0   ->  1
  elapsed       0   ->  60 วินาที
```

ของที่ได้มาคือ crew_key มาจากฟิลด์ grants_object ของมารา วอสส์ ใน story.json และมันคือกุญแจที่ปลดล็อกตู้ในตัวอย่างแรก ทั้งสองตัวอย่างจึงต่อกันเป็นเส้นทางเดียว

12. การทดสอบ ยืนยันว่าชั้นนี้ทำงานถูกจริง

ชั้น symbolic มีเทส 50 ข้อใน tests/test_engine.py ทั้งหมด **รันได้โดยไม่ต้องมี API key** เพราะไฟล์นี้ไม่แตะ LLM เลย แบ่งได้เป็นสี่กลุ่ม

กลุ่ม	ตรวจอะไร	ตัวอย่าง
กฎรายข้อ	แต่ละกฎปฏิเสธในสถานการณ์ที่ควรปฏิเสธ และอนุญาตในสถานการณ์ที่ควรอนุญาต	สอบปากคำคนที่อยู่คนละตู้ ต้องตกด้วย R005 และยืนตู้เดียวกันต้องผ่าน
ความสอดคล้องของโครงสร้าง	กฎทุกข้อต้องมีตัวตรวจจริง ลำดับกฎต้องไม่ถูกสลับ	R014 ต้องอยู่ก่อนกฎของ accuse ไม่งั้นข้อความบอกสาเหตุผิด
เกมเดินได้จนจบ	เดินตามเส้นทางเฉลย 29 ขั้นใน walkthrough.json ต้องผ่านทุกขั้นและปิดคดีได้ในเวลา	กันกฎที่ขัดกันเองจนเกมตัน ซึ่งเคยเกิดขึ้นจริงตอนเพิ่ม R005
คุณภาพของฉาก	ไล่ความสัมพันธ์ใน story.json เอง แล้วฟ้องถ้าฉากมีทางตัน	หลักฐานที่ปลดล็อกของในตู้อื่น คำบรรยายต้องเอ่ยชื่อตู้นั้นด้วย

กลุ่มที่สามคุ้มค่าที่สุด ตอนเพิ่มกฎ R005 ให้คุมการสอบปากคำ เทสกลุ่มนี้ล้มทันที 11 ข้อ เพราะเส้นทางเฉลยเดิมแอบอาศัยช่องโหว่ที่สอบปากคำข้ามตู้ได้อยู่ ถ้าไม่มีเทสชุดนี้ เกมจะเล่นไม่จบโดยไม่มีใครรู้ จนกว่าจะมีคนเล่นถึงจุดนั้น

กลุ่มที่สี่ก็จับบั๊กได้จริง มันไล่ทุกความสัมพันธ์ requires_evidence เองแล้วพบว่า มีหลักฐานสองชิ้นที่ปลดล็อกของในตู้อื่นโดยไม่บอกว่าตู้ไหน ซึ่งเป็นทางตันที่มองด้วยตาไม่เห็น

13. สรุปสิ่งที่ควรจำ

ประเด็น	รายละเอียด
ไม่มี LLM ในชั้นนี้	engine.py ไม่ import อะไรจาก app/neural บังคับด้วยโครงสร้าง ไม่ใช่แค่ข้อตกลง
กฎเป็นข้อมูล	กฎ 15 ข้ออยู่ใน rules.json แก้ลำดับ เงื่อนไข หรือข้อความได้โดยไม่แตะโค้ด
ฉากเป็นข้อมูล	โลกทั้งใบอยู่ใน story.json เปลี่ยนคดีใหม่ได้โดยไม่แตะโค้ด
สายเหตุผลซื่อตรง	reasoning_chain ถูกสร้างระหว่างวนลูปตรวจกฎ จึงตรงกับสิ่งที่ทำจริงเสมอ
ตัดสินแยกจากเปลี่ยนสถานะ	decide() บอกว่าควรเปลี่ยนอะไร apply_changes() เป็นคนเปลี่ยนจริง และคืนก้อนใหม่เสมอ
สิ่งที่เห็นเท่ากับสิ่งที่ทำได้	look_around และ offerable ถามจากกฎชุดเดียวกับที่ใช้ตัดสิน ไม่มีรายการเขียนมือแยก
เวลาเดินนอกคำตัดสิน	เพื่อให้เปรียบเทียบสองโหมดของการทดลองได้อย่างเป็นธรรม